

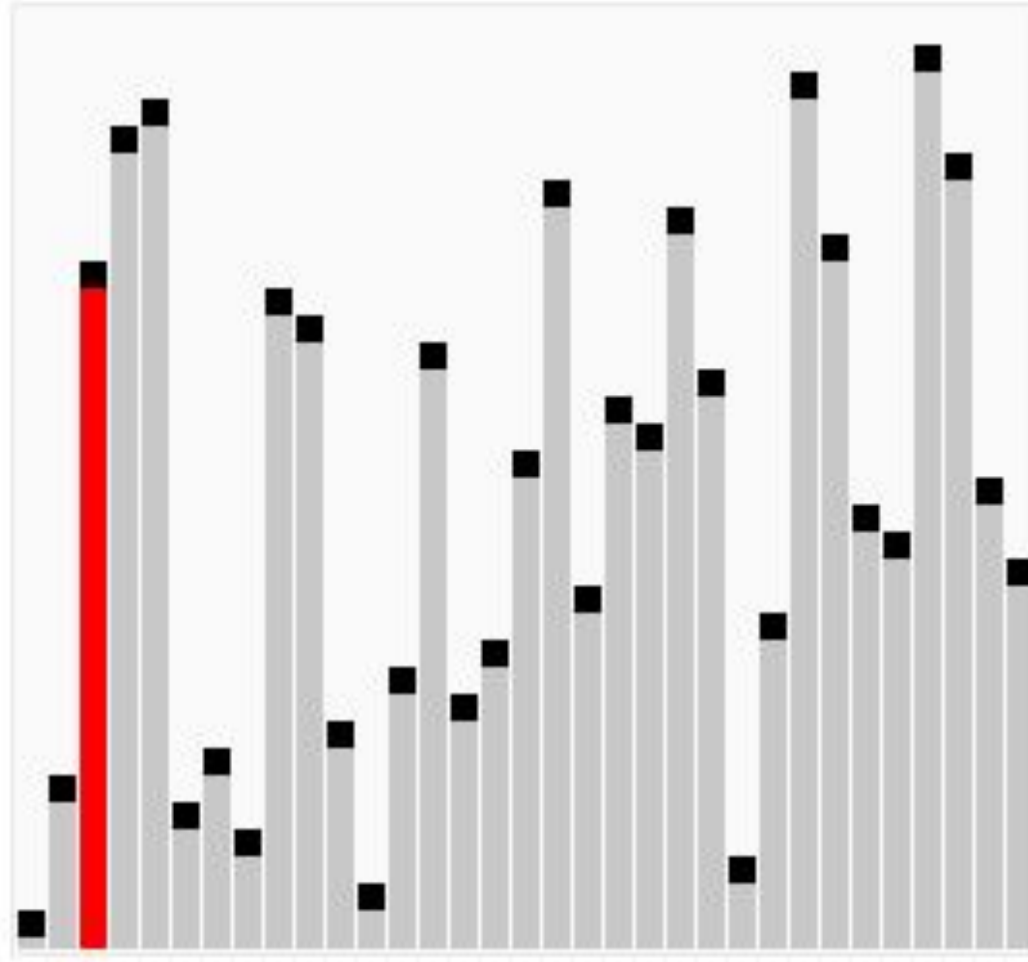
Introduction to Algorithms

Levon Prazyan

Course Overview

- Introduction
- **Sorting and Searching**
- Arrays and Lists
- Stacks and Queues, Double Ended Queue
- Heaps
- Trees
- Hash Tables
- Dynamic Programming
- Greedy Algorithms
- Graph Algorithms
- Network Flow
- NP-Completeness

Bubble Sort Illustration



Bubble Sort

6 5 3 1 8 7 2 4

Bubble Sort

BubbleSort(A):

```
for i = 1 to A.length:
```

```
swapped = false
```

```
for j = 1 to A.length-i
```

```
if A[j] > A[j+1]
```

```
swap(A[j], A[j+1])
```

swapped = true

```
if swapped == false:
```

break // exit early

	Pass 1	Pass 2	Pass 3	Pass 4	Pass 5	Pass 6	Pass 7	Pass 8	Pass 9
703	908	908	908	908	908	908	908	908	908
765	703	897	897	897	897	897	897	897	897
677	765	703	765	765	765	765	765	765	765
612	677	765	703	703	703	703	703	703	703
509	612	677	677	677	677	677	677	677	677
154	509	612	653	653	653	653	653	653	653
426	154	509	612	612	612	612	612	612	612
653	426	154	509	512	512	512	512	512	512
275	653	426	154	509	509	509	509	509	509
897	275	653	426	154	503	503	503	503	503
170	897	275	512	426	154	426	426	426	426
908	170	512	275	503	426	154	275	275	275
061	512	170	503	275	275	275	154	170	170
512	061	503	170	170	170	170	170	154	154
087	503	061	087	087	087	087	087	087	087
503	087	087	061	061	061	061	061	061	061

Bubble Sort: Correctness

BubbleSort(A):

```
for i = 1 to A.length:
    swapped = false
    for j = 1 to A.length-i
        if A[j] > A[j+1]
            swap(A[j], A[j+1])
            swapped = true
    if swapped == false:
        break // exit early
```

Invariant: At the end of the i -th outer loop iteration, the last i elements of the array are in their correct sorted positions.

Bubble Sort: Correctness: Proof by Induction

- **Base Case:** Before the first iteration, no elements are sorted.
- **Inductive Step:** Assume that at the end of iteration i , the last i elements are sorted.
 - In iteration $i + 1$, the inner loop goes through the unsorted part of the array.
 - The largest element in this part bubbles up to its correct position at index $n - i - 1$ by swapping adjacent elements when needed.
 - This ensures that after iteration $i + 1$, the last $i + 1$ elements are sorted.
 - By induction, after $n-1$ iterations, the entire array is sorted
- **Termination:** The algorithm terminates because
 - The outer loop runs at most $n - 1$ times.
 - The inner loop ensures that no further swaps occur if the array becomes sorted early.
 - The total number of comparisons and swaps is finite.

Bubble Sort: Complexity (1)

BubbleSort(A):

for i = 1 to A.length:

swapped = false

for j = 1 to A.length-i

if A[j] > A[j+1]

swap(A[j], A[j+1])

swapped = true

if swapped == false:

break // exit early

Time for worst case

$n * c_1$

$n * c_2$

$\sum_{i=1}^n (n - i) * c_3$

$\sum_{i=1}^n (n - i) * c_4$

$\sum_{i=1}^n (n - i) * c_5$

$\sum_{i=1}^n (n - i) * c_6$

$n * c_7$

c_8

Bubble Sort: Complexity (2)

$$\dot{T}(n) = n * (c_1 + c_2 + c_7) + (c_3 + c_4 + c_5 + c_6) * \sum_{i=1}^n (n - i) + c_8$$

$$T(n) = an^2 + bn + c$$

Worst case: $T(\mathbf{n}) = \mathbf{\Theta}(\mathbf{n}^2)$

Best case: $T(\mathbf{n}) = \mathbf{O}(\mathbf{n})$

Bubble Sort: Usage

When use

- Input is nearly sorted
- Input size is small
- Limited memory
- Stability is important

When avoid

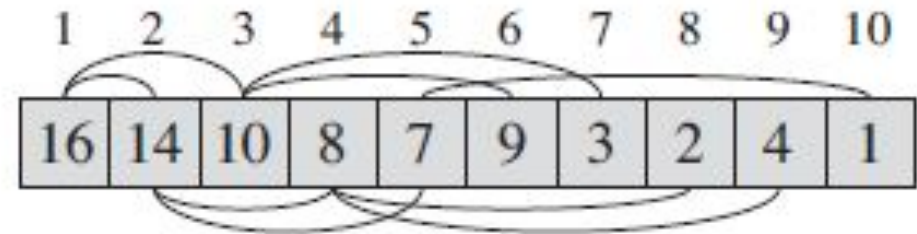
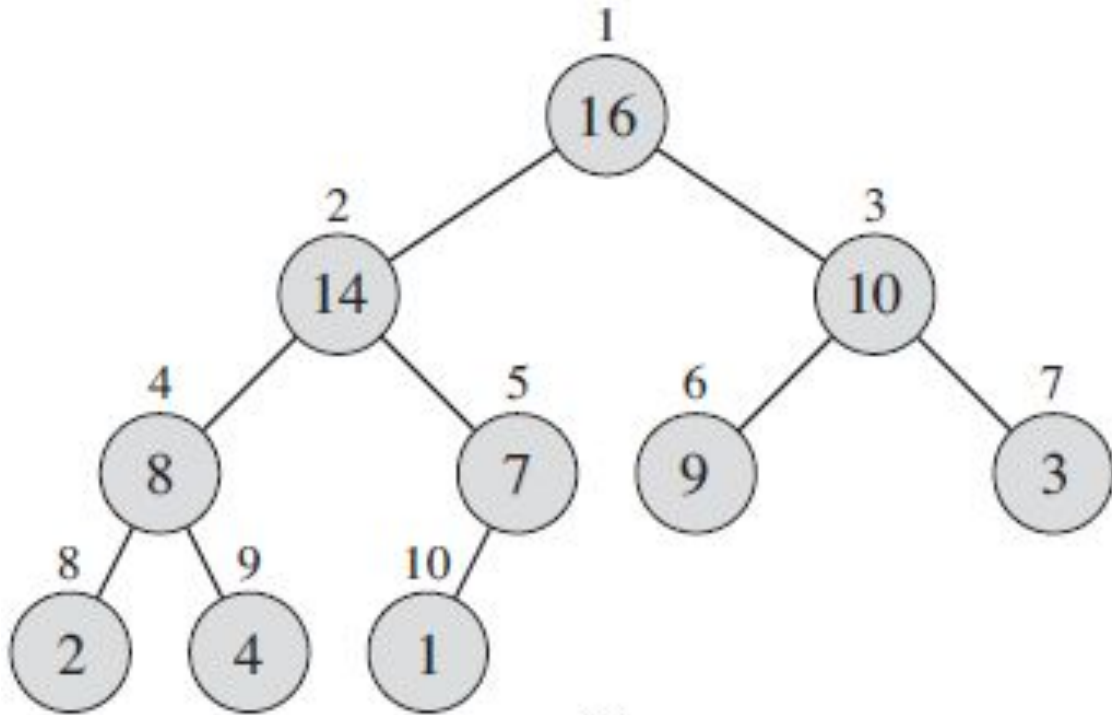
- Input size is large
- Performance is a priority
- General-purpose

Heap Sort

Heap, Priority Queue

Heap

The (binary) heap data structure is an array object that we can view as a nearly complete binary tree.



Heap Property

PARENT(*i*)

1 **return** $\lfloor i/2 \rfloor$

LEFT(*i*)

1 **return** $2i$

RIGHT(*i*)

1 **return** $2i + 1$

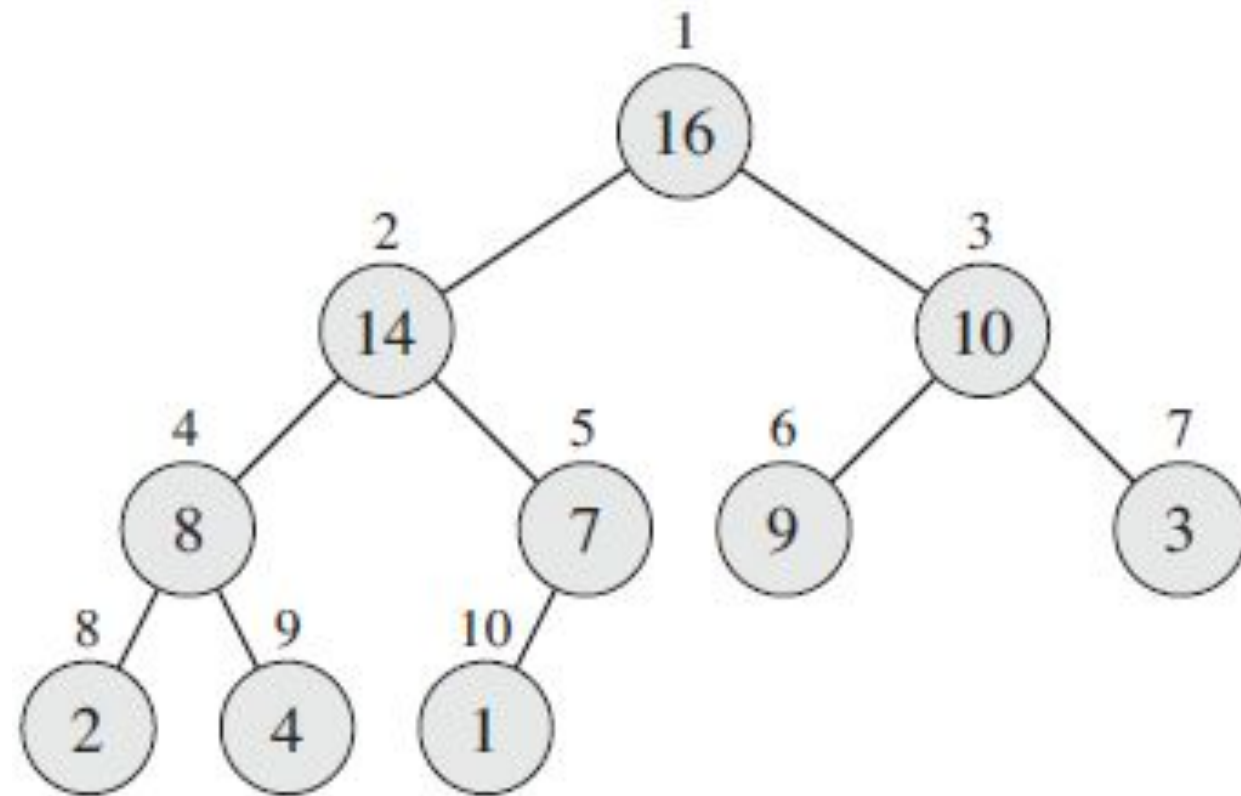
- Max-heap property

$$A[\text{Parent}(i)] \geq A(i)$$

- Min-heap property

$$A[\text{Parent}(i)] \leq A(i)$$

Heap Height



- The **height** of a node in a heap is the number of edges on the longest simple downward path from the node to a leaf
- The **height** of the heap to be the height of its root.
- Since a heap of n elements is based on a complete binary tree, its height is $\Theta(\log n)$

Heap: Operations

- The MAX-HEAPIFY procedure, which runs in $O(\log n)$ time, is the key to maintaining the max-heap property.
- The BUILD-MAX-HEAP procedure, which runs in $O(n)$ time, produces a maxheap from an unordered input array.
- The HEAPSORT procedure, which runs in $O(n \log n)$ time, sorts an array in place.
- The MAX-HEAP-INSERT, HEAP-EXTRACT-MAX, HEAP-INCREASE-KEY, and HEAP-MAXIMUM procedures, which run in $O(\log n)$ time, allow the heap data structure to implement a priority queue.

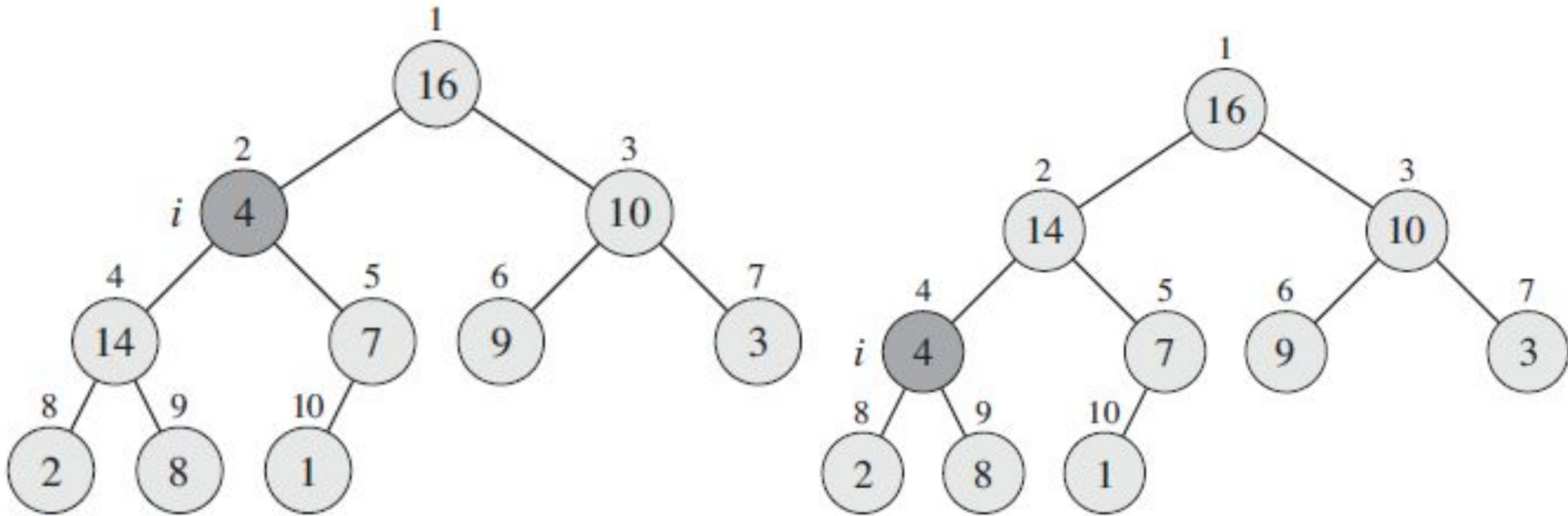
Heap Sort: Max-Heapify (1)

MAX-HEAPIFY(A, i)

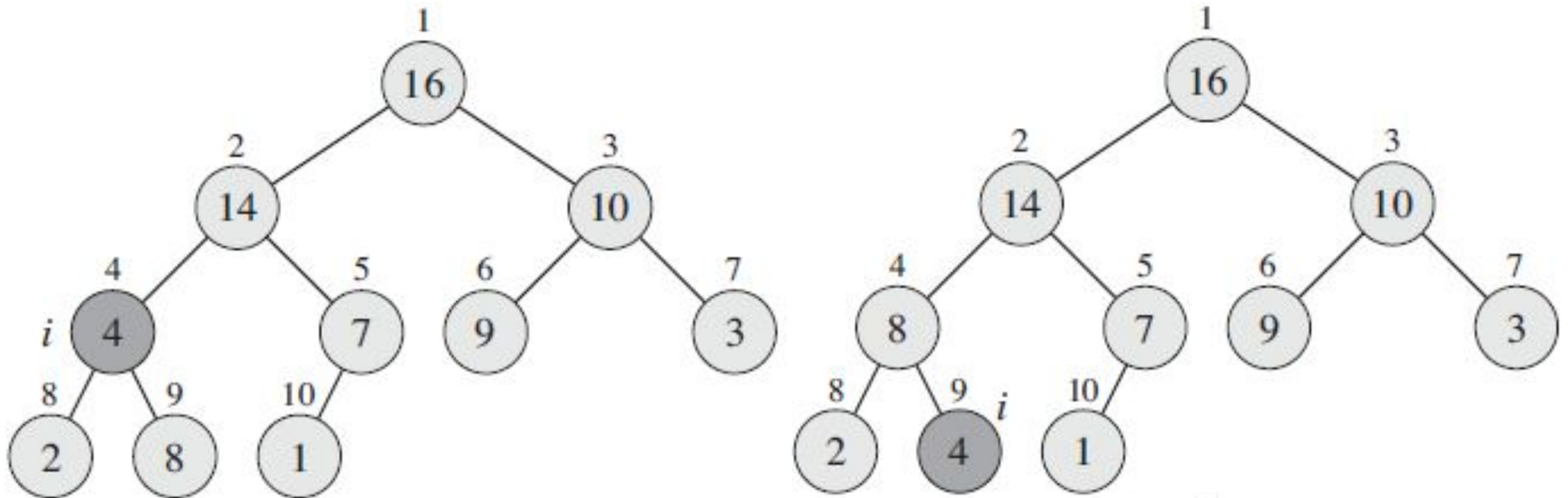
```
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$ 
4       $largest = l$ 
5  else  $largest = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] > A[largest]$ 
7       $largest = r$ 
8  if  $largest \neq i$ 
9      exchange  $A[i]$  with  $A[largest]$ 
10     MAX-HEAPIFY( $A, largest$ )
```

- MAX-HEAPIFY assumes that the binary trees rooted at LEFT(i) and RIGHT(i) are max-heaps, but that $A(i)$ might be smaller than its children.

Heap Sort: Max-Heapify (2)



Heap Sort: Max-Heapify (3)



Heap Sort: Max-Heapify Running Time (1)

MAX-HEAPIFY(A, i)

1	$l = \text{LEFT}(i)$	$\theta(1) +$
2	$r = \text{RIGHT}(i)$	$\theta(1) +$
3	if $l \leq A.\text{heap-size}$ and $A[l] > A[i]$	$\theta(1) +$
4	$\text{largest} = l$	$\theta(1) +$
5	else $\text{largest} = i$	$\theta(1) +$
6	if $r \leq A.\text{heap-size}$ and $A[r] > A[\text{largest}]$	$\theta(1) +$
7	$\text{largest} = r$	$\theta(1) +$
8	if $\text{largest} \neq i$	$\theta(1) +$
9	exchange $A[i]$ with $A[\text{largest}]$	$\theta(1) +$
10	MAX-HEAPIFY($A, \text{largest}$)	$T(2n/3)$

Worst case running time $T(n) =$

$\theta(1) +$

$\theta(1) +$

$\theta(1) +$

$\theta(1) +$

$\theta(1) +$

$\theta(1) +$

$\theta(1) +$

$\theta(1) +$

$\theta(1) +$

$\theta(1) +$

$T(2n/3)$

Heap Sort: Max-Heapify Running Time (2)

- $$T(n) = T(2n/3) + \Theta(1)$$

$$\Theta(1) = \Theta\left(n^{\log_{\frac{2}{3}} 1}\right)$$

$$T(n) = \Theta\left(n^{\log_{\frac{2}{3}} 1} \log n\right)$$

Worst case: $T(n) = \Theta(\log n)$

In general: $T(n) = O(\log n)$

Heap Sort: Build-Max-Heap

- The Build-Max-Heap produces a maxheap from an unordered input array

BUILD-MAX-HEAP(*A*)

1 *A.heap-size* = *A.length*

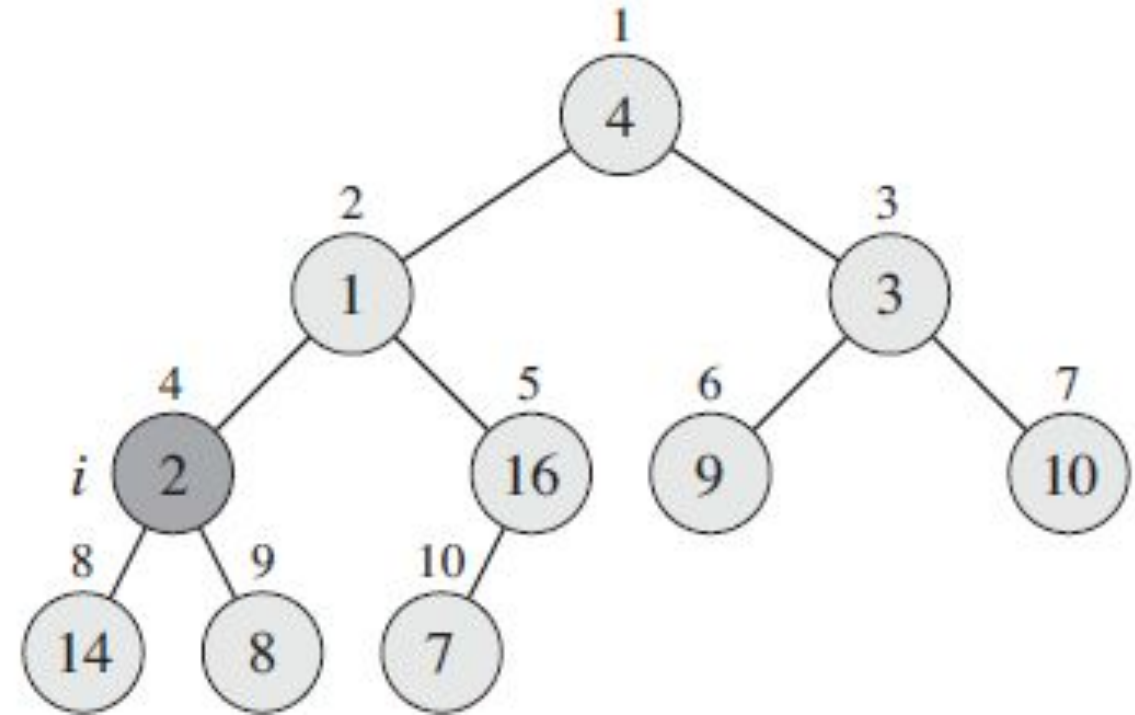
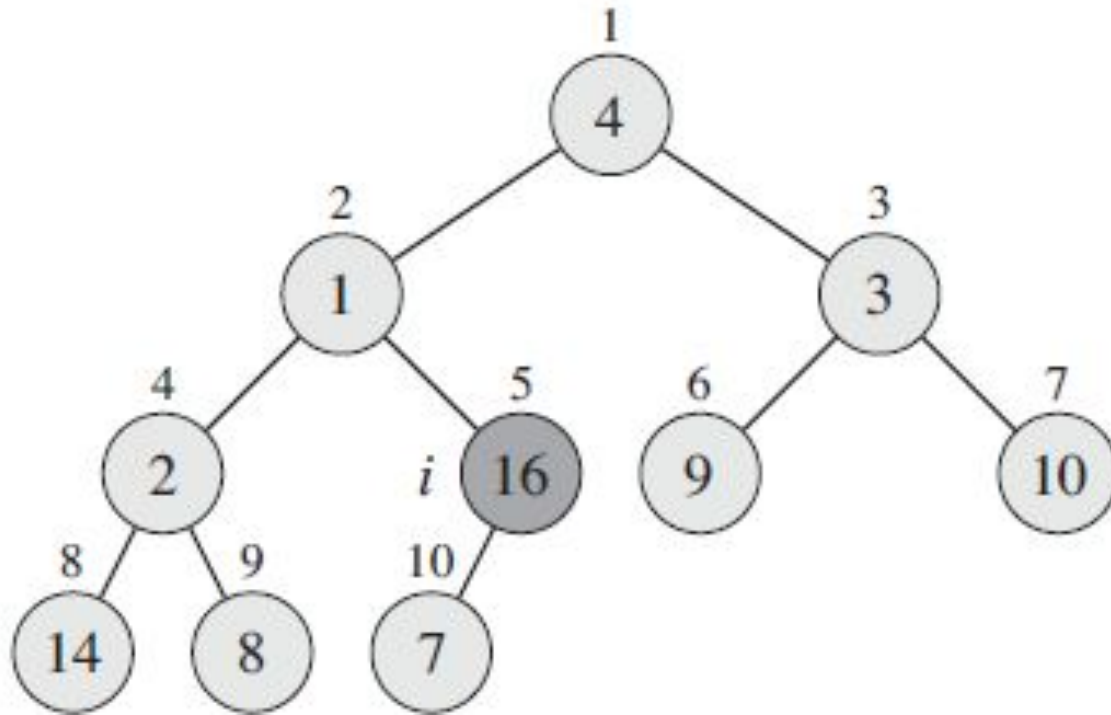
2 **for** *i* = $\lfloor A.length/2 \rfloor$ **downto** 1

3 **MAX-HEAPIFY**(*A*, *i*)

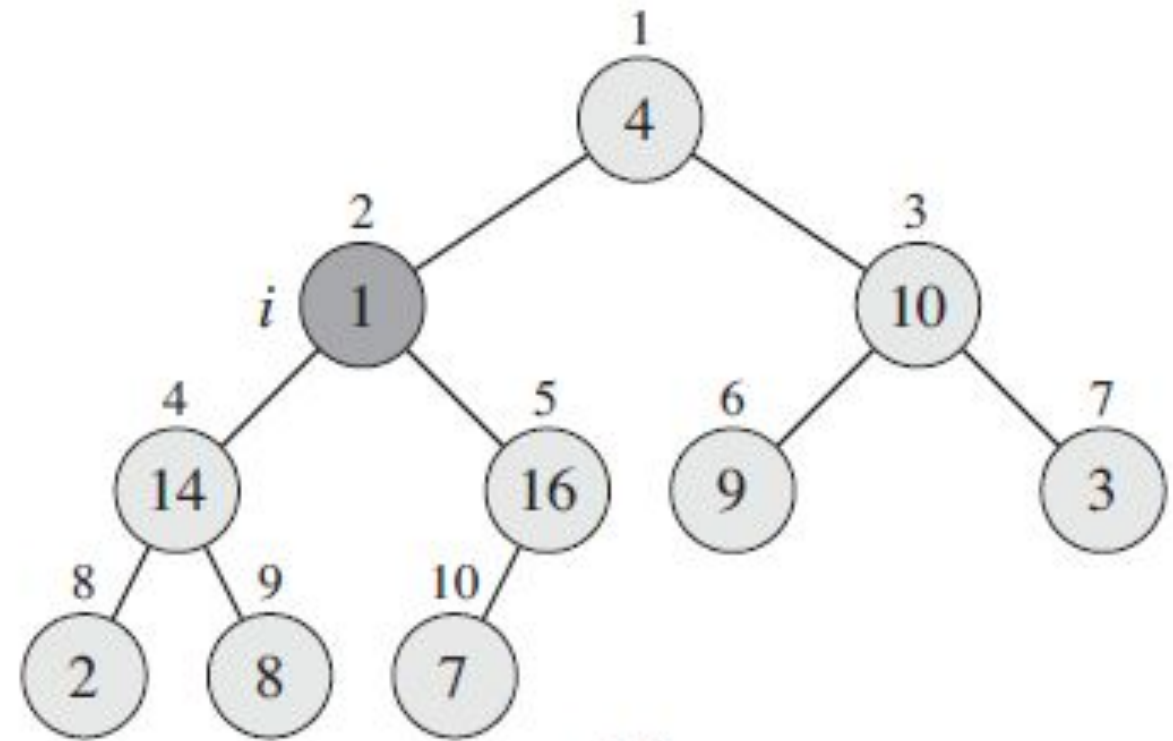
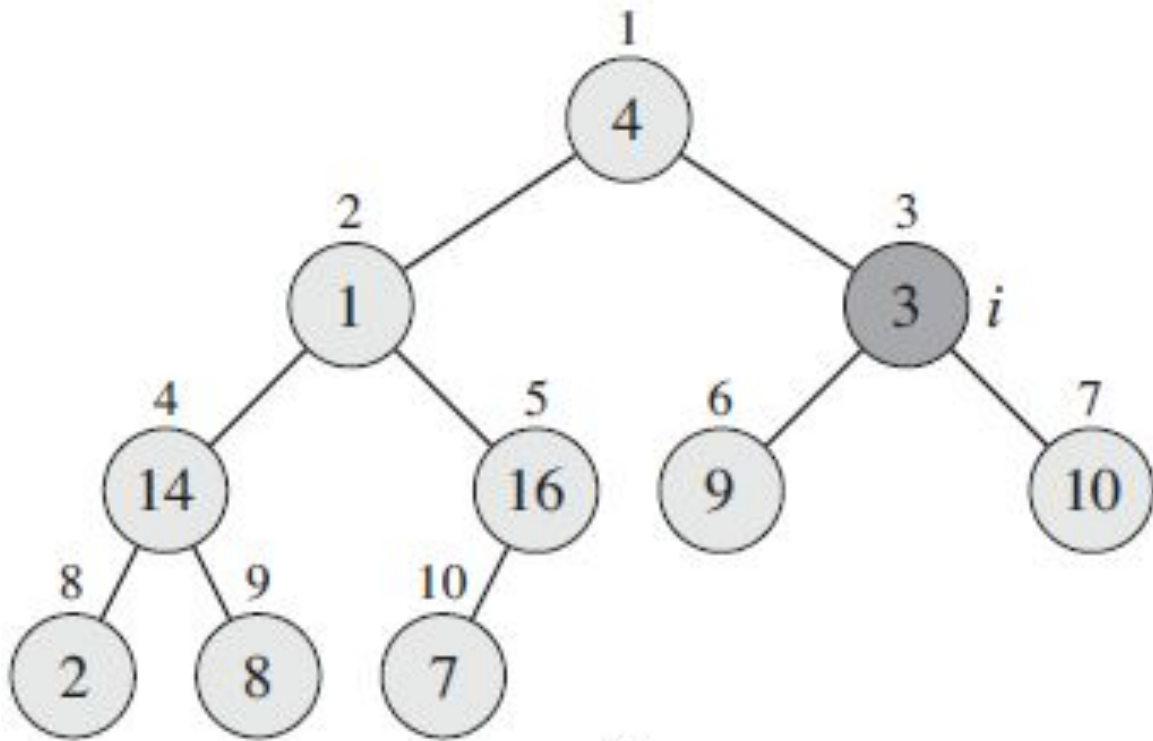
Heap Sort: Build-Max-Heap (2)

A

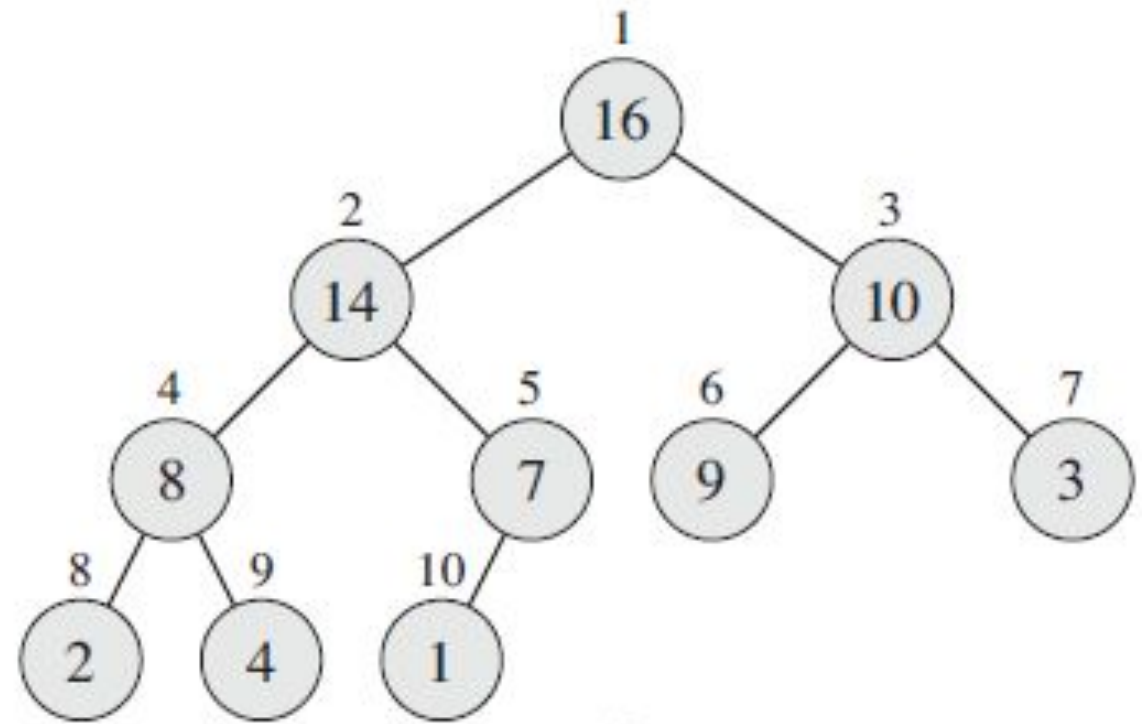
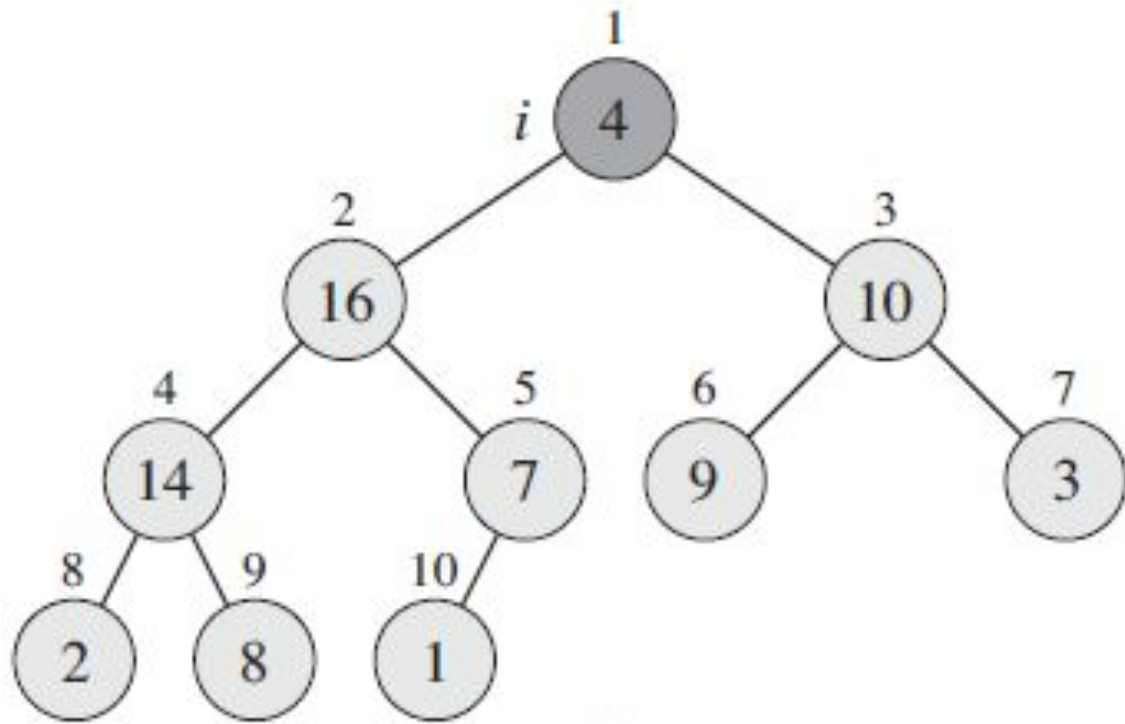
4	1	3	2	16	9	10	14	8	7
---	---	---	---	----	---	----	----	---	---



Heap Sort: Build-Max-Heap (3)



Heap Sort: Build-Max-Heap (4)



Heap Sort: Build-Max-Heap: Correctness (1)

BUILD-MAX-HEAP(A)

```
1   $A.heap-size = A.length$   
2  for  $i = \lfloor A.length/2 \rfloor$  downto 1  
3      MAX-HEAPIFY( $A, i$ )
```

loop invariant: At the start of each iteration of the **for** loop of, each node $i + 1; i + 2; \dots; n$ is the root of a max-heap.

Heap Sort: Build-Max-Heap: Correctness (2)

- **Initialization:** All nodes at indexes $n/2 + 1$; $n/2 + 2$; ... ; n are leaf nodes, thus have no child nodes and are max-heaps
- **Maintenance:** The children of node i are numbered higher than i . By the loop invariant, they are both roots of max-heaps. The call of the Max-Heapify makes a heap with root i a max-heap.
- **Termination:** At termination ($i == 0$). each node $1, 2, \dots, n$ is the root of a max-heap. In particular node 1 is.

Heap Sort: Build-Max-Heap: Running Time (1)

BUILD-MAX-HEAP(*A*)

```
1  A.heap-size = A.length
2  for i =  $\lfloor A.length/2 \rfloor$  downto 1
3      MAX-HEAPIFY(A, i)
```

Worst case running time $T(n) =$

$\Theta(1) +$

$\Theta(n/2) +$

$\sum_{i=1}^{\log n} \frac{n * i}{2^i}$

Heap Sort: Build-Max-Heap: Running Time (2)

-

$$T(n) = \Theta(n) + \sum_{i=1}^{\log n} \frac{n * i}{2^i}$$
$$\sum_{i=1}^{\log n} \frac{i}{2^i} = 2 - \frac{\log n + 2}{n} < 2$$

$$\mathbf{T(n) = O(n)}$$

Heap Sort

HEAPSORT(*A*)

```
1  BUILD-MAX-HEAP(A)
2  for i = A.length downto 2
3      exchange A[1] with A[i]
4      A.heap-size = A.heap-size - 1
5      MAX-HEAPIFY(A, 1)
```

• Total time: $T(n) =$

$O(n) +$

$\Theta(n) +$

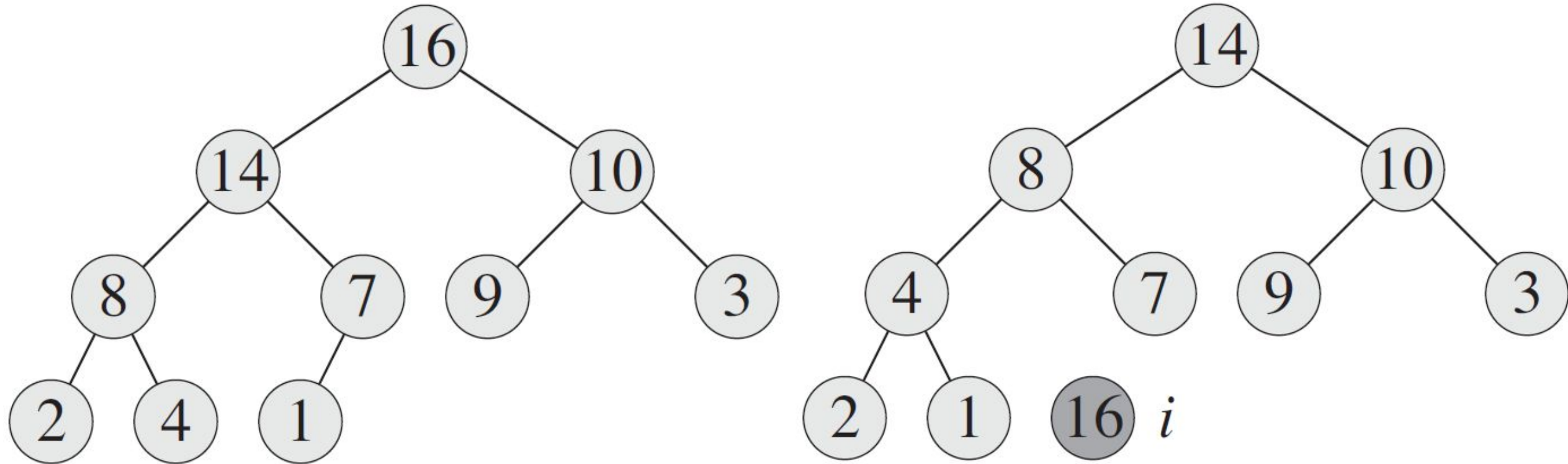
$\Theta(n) +$

$\Theta(n) +$

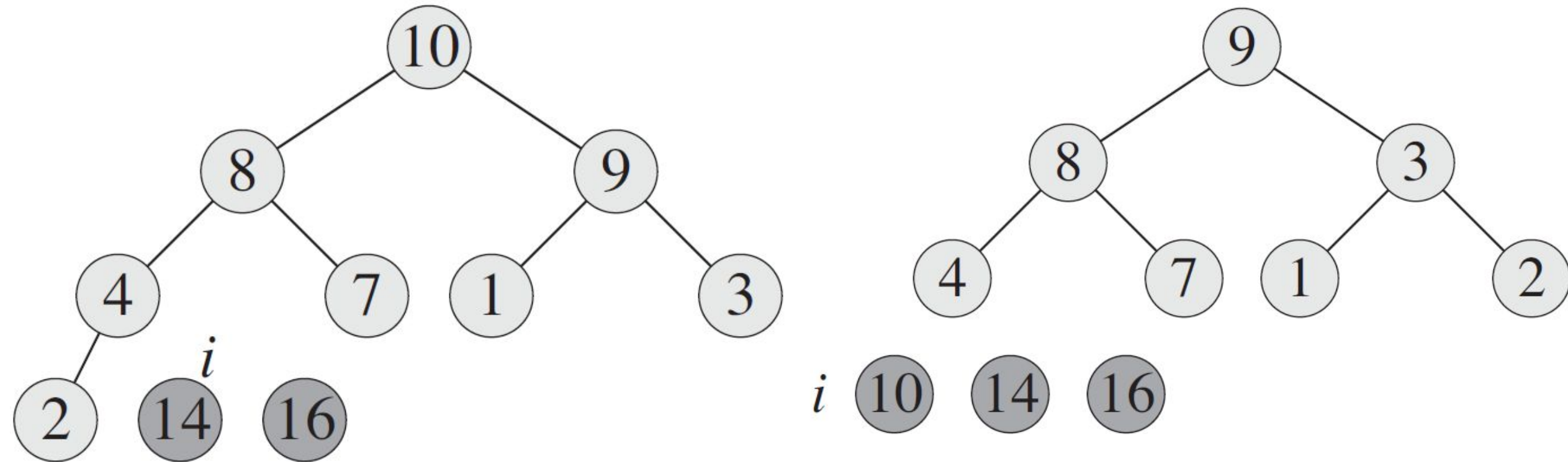
$O(n * \log n)$

$T(n) = O(n * \log n)$

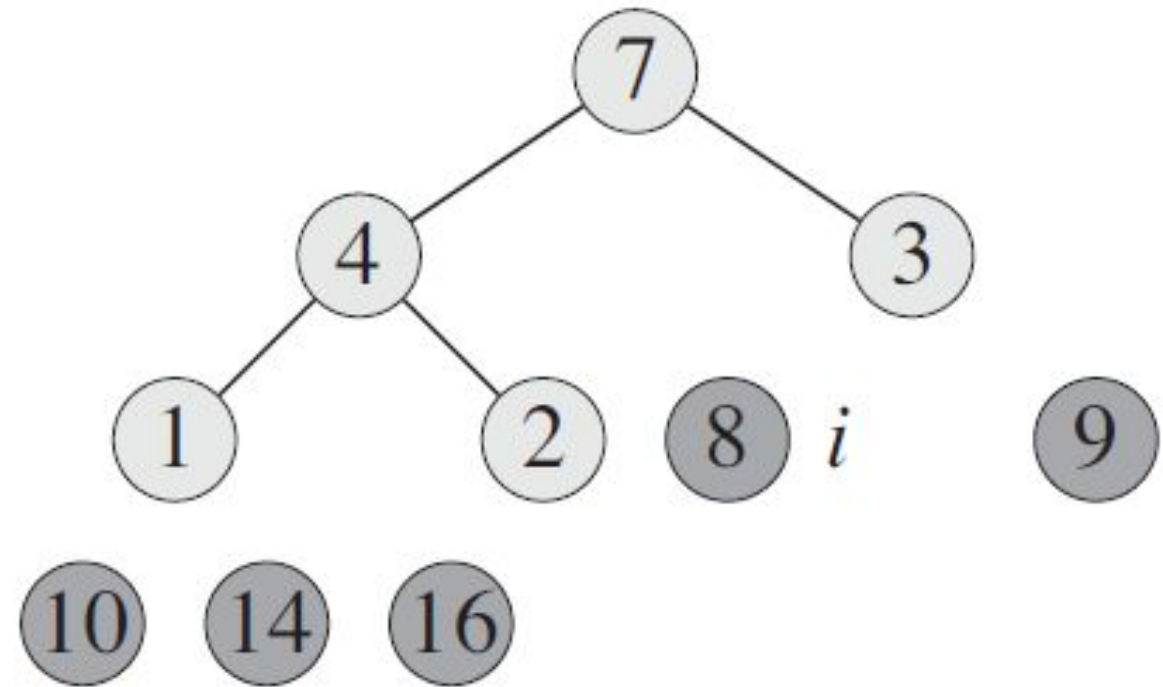
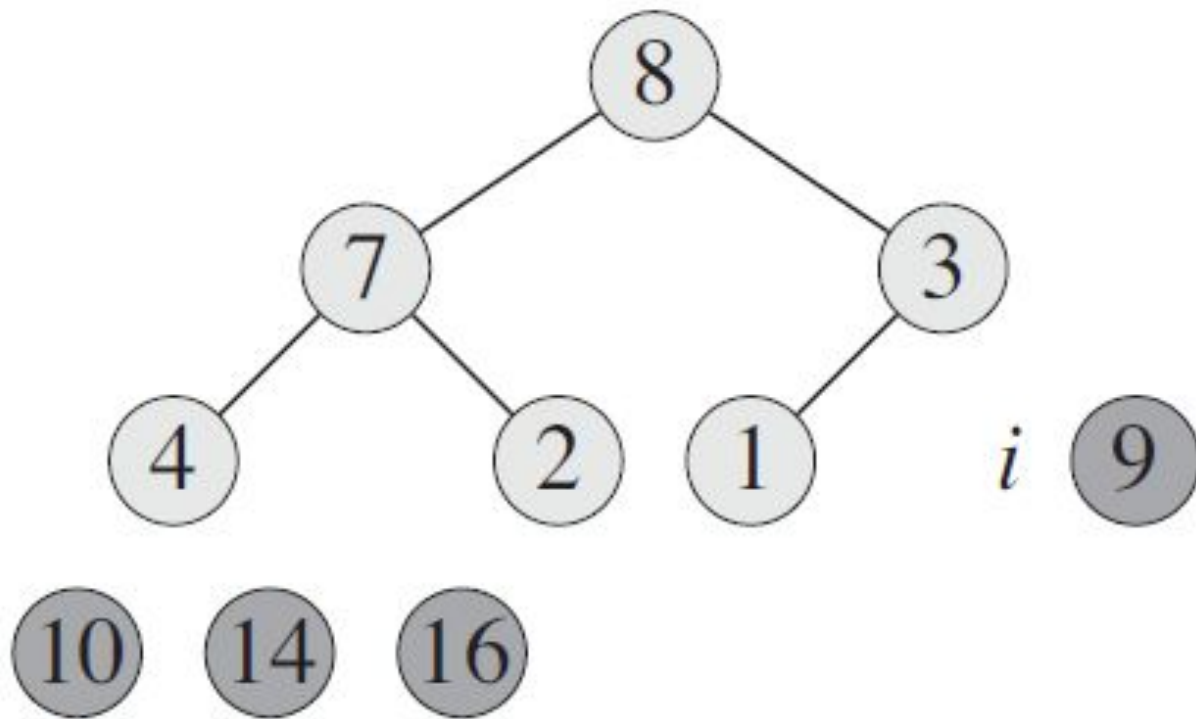
Heap Sort: Example (1)



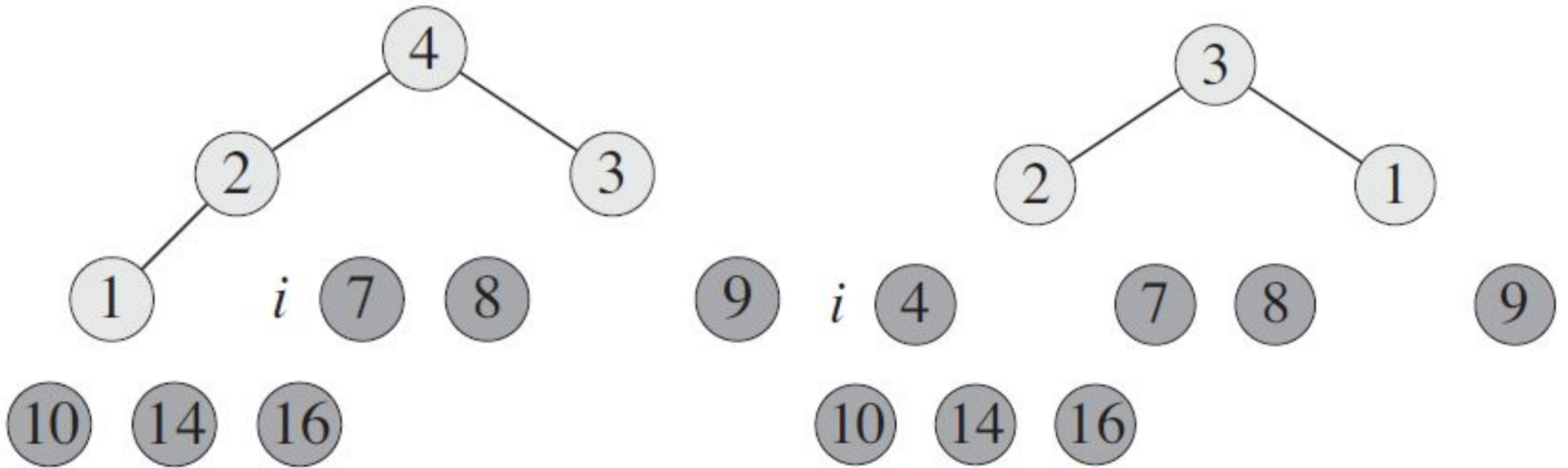
Heap Sort: Example (2)



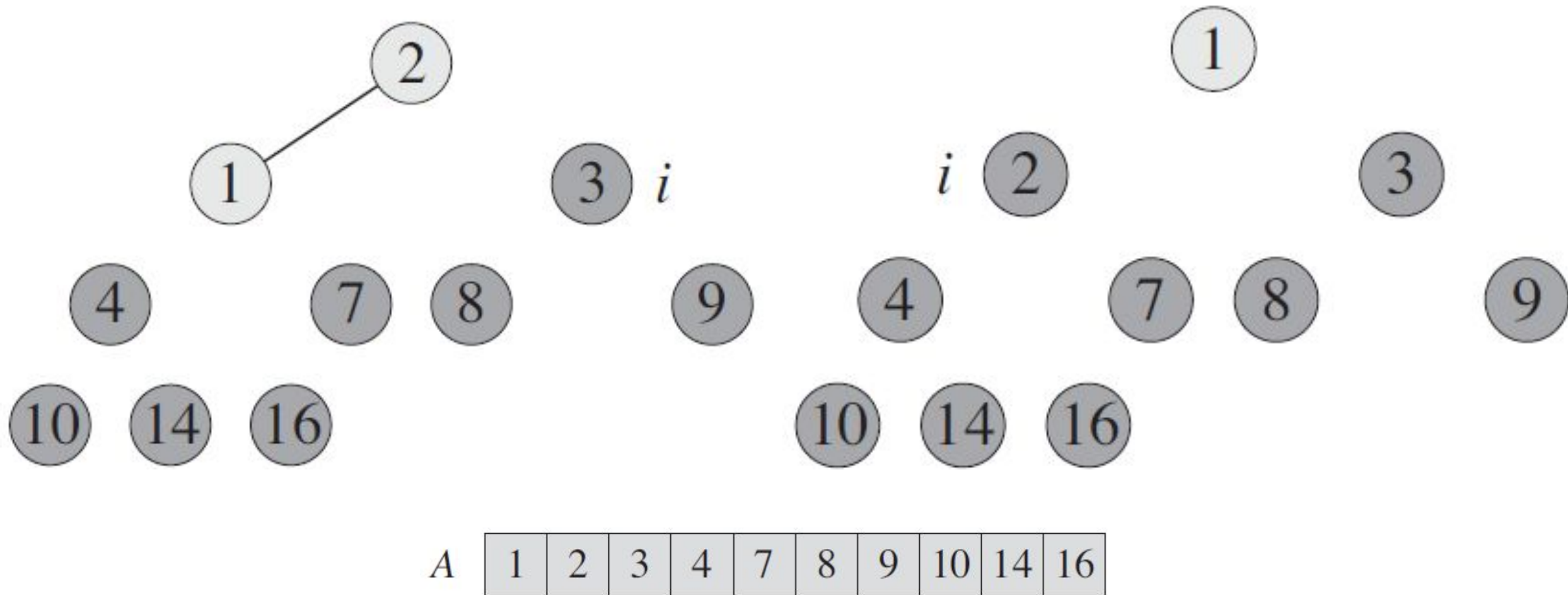
Heap Sort: Example (3)



Heap Sort: Example (4)



Heap Sort: Example (4)



Priority Queue (1)

A **priority queue** is a data structure for maintaining a set S of elements, each with an associated value called a key.

- Insert
- Maximum
- Extract-Max
- Increase-Key

Priority Queue (2)

HEAP-MAXIMUM(A)

```
1  return  $A[1]$ 
```

Total time: $\Theta(1)$

HEAP-EXTRACT-MAX(A)

```
1  if  $A.heap-size < 1$   
2      error "heap underflow"  
3   $max = A[1]$   
4   $A[1] = A[A.heap-size]$   
5   $A.heap-size = A.heap-size - 1$   
6  MAX-HEAPIFY( $A, 1$ )  
7  return  $max$ 
```

Total time: $O(\log n)$

Priority Queue (3)

HEAP-INCREASE-KEY(A, i, key)

```
1  if  $key < A[i]$ 
2      error "new key is smaller than current key"
3   $A[i] = key$ 
4  while  $i > 1$  and  $A[PARENT(i)] < A[i]$ 
5      exchange  $A[i]$  with  $A[PARENT(i)]$ 
6       $i = PARENT(i)$ 
```

Total time: $O(\log n)$

MAX-HEAP-INSERT(A, key)

```
1   $A.heap-size = A.heap-size + 1$ 
2   $A[A.heap-size] = -\infty$ 
3  HEAP-INCREASE-KEY( $A, A.heap-size, key$ )
```

Total time: $O(\log n)$

Build Max Heap

BUILD-MAX-HEAP(A)

```
1   $A.heap-size = A.length$   
2  for  $i = \lfloor A.length/2 \rfloor$  downto 1  
3      MAX-HEAPIFY( $A, i$ )
```

BUILD-MAX-HEAP'(A)

```
1   $A.heap-size = 1$   
2  for  $i = 2$  to  $A.length$   
3      MAX-HEAP-INSERT( $A, A[i]$ )
```

Quick Sort

QuickSort, RandomizedQuickSort

Quicksort

Worst-case running time:

Average-case running time:

Quicksort: Devide-and-Conquer

- **Divide:** Partition $A[p..r]$ into two subarrays $A[p..q-1]$ and $A[q+1..r]$ such
 $A[i] \leq A[q]; p \leq i \leq q-1$ and
 $A[q] < A[i]; q+1 \leq i \leq r$
- **Conquer:** Sort the subarrays $A[p..q-1]$ and $A[q+1..r]$ by recursive calls to quicksort
- **Combine:** No work is needed to combine

Quicksort Procedure

QUICKSORT(A, p, r)

1 **if** $p < r$

2 *// Partition the subarray around the pivot, which ends up in $A[q]$.*

3 $q = \text{PARTITION}(A, p, r)$

4 QUICKSORT($A, p, q - 1$) *// recursively sort the low side*

5 QUICKSORT($A, q + 1, r$) *// recursively sort the high side*

Quicksort: Partitioning procedure

PARTITION(A, p, r)

```
1   $x = A[r]$  // the pivot
2   $i = p - 1$  // highest index into the low side
3  for  $j = p$  to  $r - 1$  // process each element other than the pivot
4      if  $A[j] \leq x$  // does this element belong on the low side?
5           $i = i + 1$  // index of a new slot in the low side
6          exchange  $A[i]$  with  $A[j]$  // put this element there
7  exchange  $A[i + 1]$  with  $A[r]$  // pivot goes just to the right of the low side
8  return  $i + 1$  // new index of the pivot
```

Quicksort and Partitioning

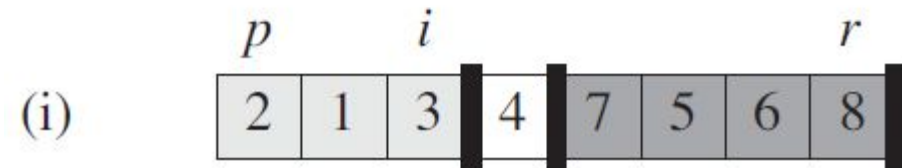
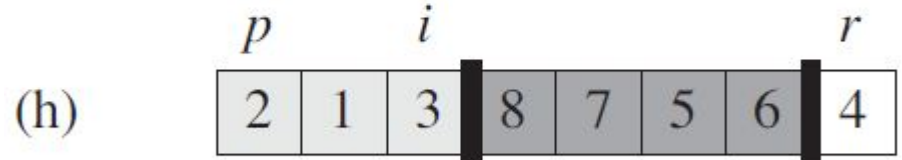
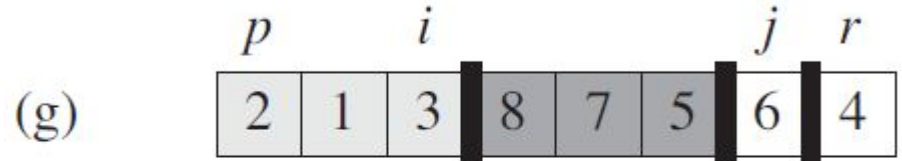
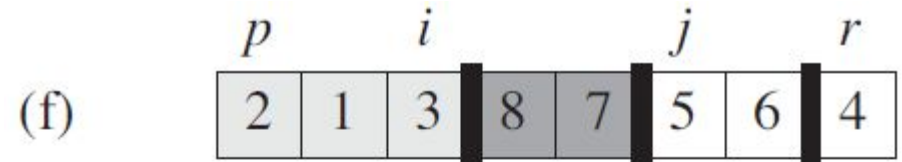
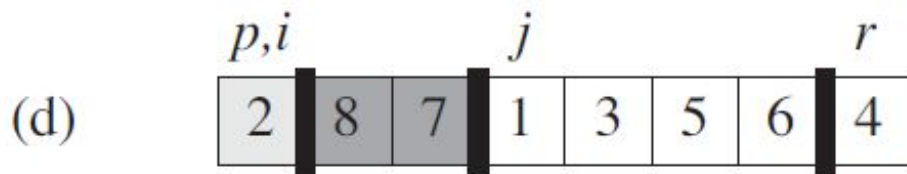
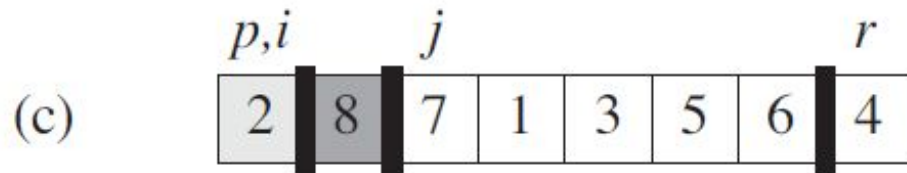
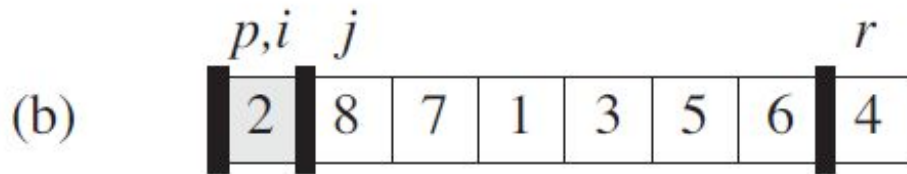
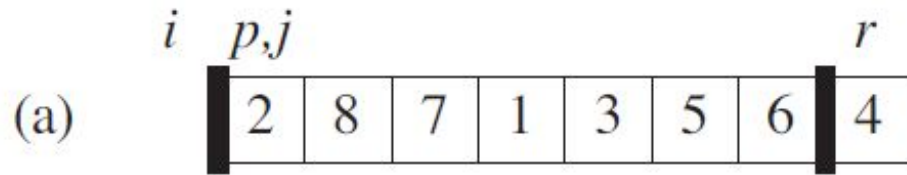
QUICKSORT(A, p, r)

```
1  if  $p < r$ 
2       $q = \text{PARTITION}(A, p, r)$ 
3      QUICKSORT( $A, p, q - 1$ )
4      QUICKSORT( $A, q + 1, r$ )
```

PARTITION(A, p, r)

```
1   $x = A[r]$ 
2   $i = p - 1$ 
3  for  $j = p$  to  $r - 1$ 
4      if  $A[j] \leq x$ 
5           $i = i + 1$ 
6          exchange  $A[i]$  with  $A[j]$ 
7  exchange  $A[i + 1]$  with  $A[r]$ 
8  return  $i + 1$ 
```

Quicksort: Partitioning



Quicksort: Partitioning: Correctness

PARTITION(A, p, r)

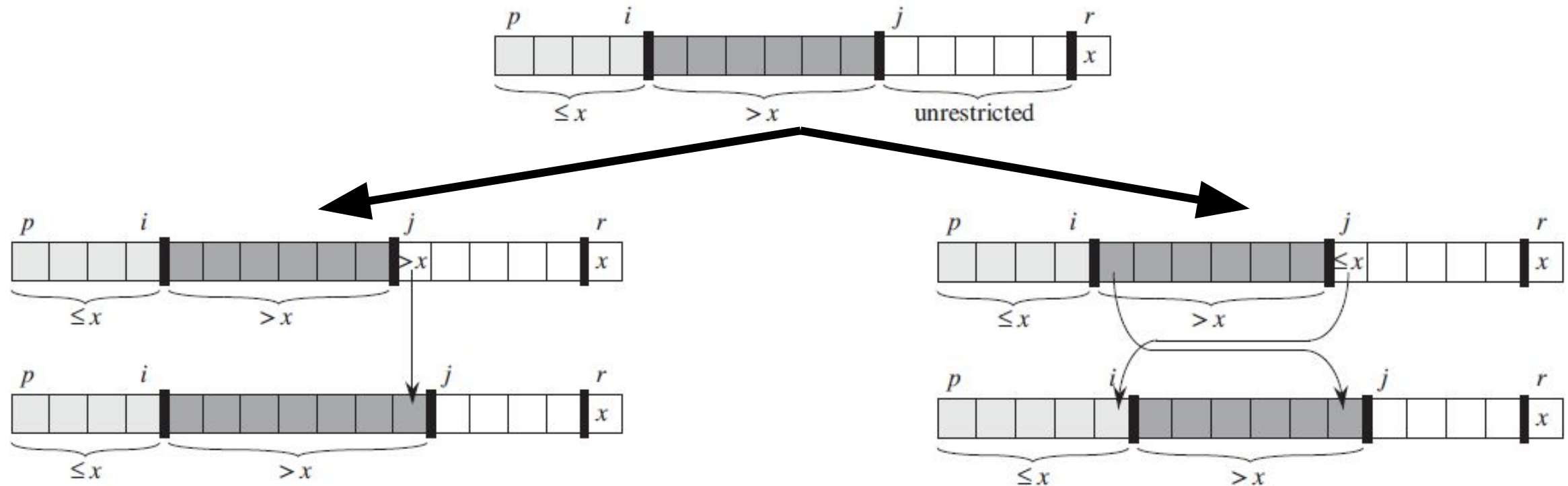
```
1   $x = A[r]$ 
2   $i = p - 1$ 
3  for  $j = p$  to  $r - 1$ 
4      if  $A[j] \leq x$ 
5           $i = i + 1$ 
6          exchange  $A[i]$  with  $A[j]$ 
7  exchange  $A[i + 1]$  with  $A[r]$ 
8  return  $i + 1$ 
```

loop invariant:

1. $A[k] \leq x ; p \leq k \leq i$
2. $A[k] > x ; i + 1 \leq k \leq j - 1$
3. $A[k] = x ; k = r$

Quicksort: Partitioning: Correctness (2)

- **Initialization:** $i = p - 1$ and $j = p$. No elements are between $[p, i]$ and $[i + 1, j - 1]$. Invariant points are satisfied.
- **Maintenance:** The assumption is correct for $j - p$ iteration.



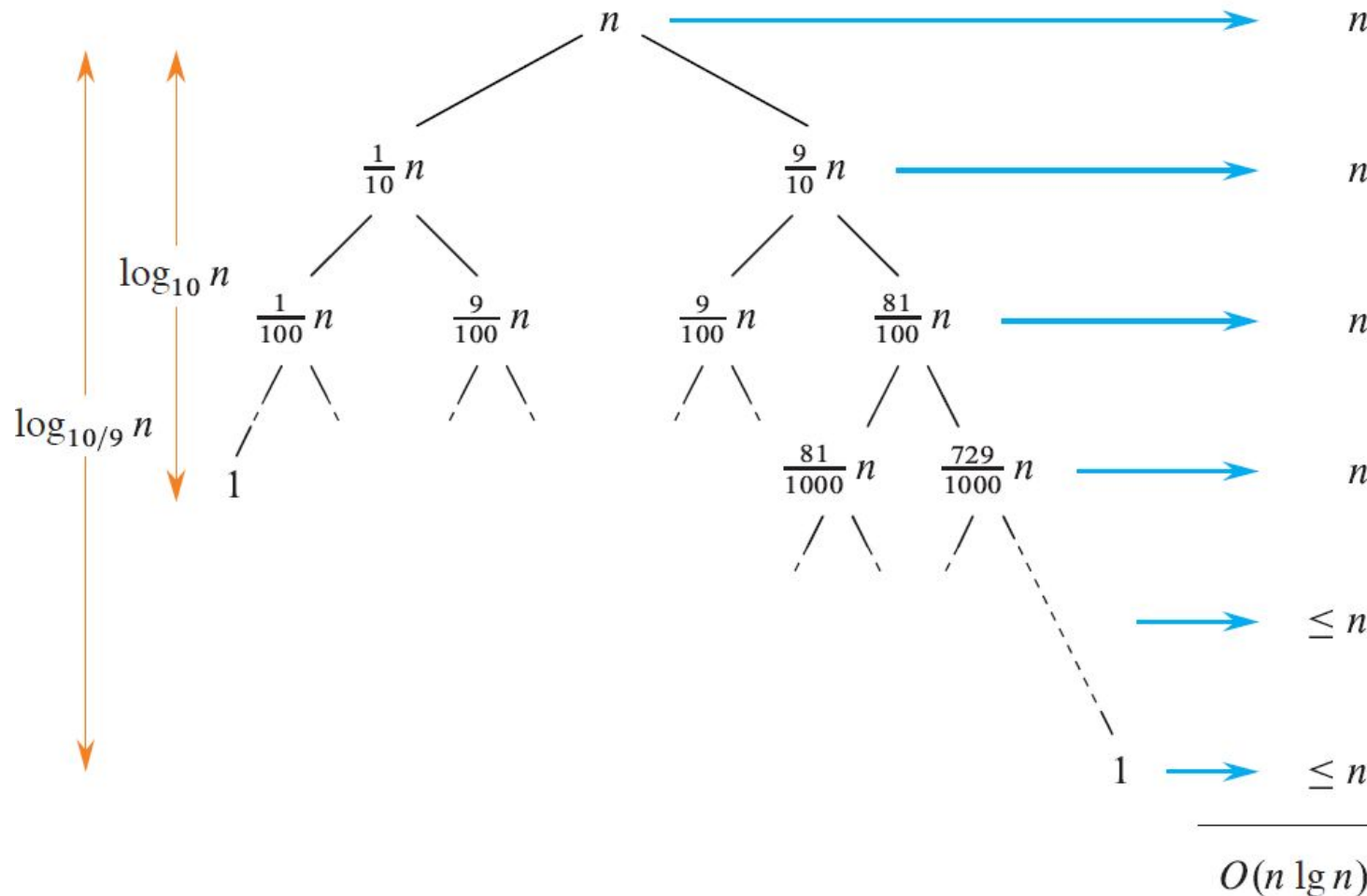
Quicksort: Partitioning: Correctness (3)

- **Termination:** At termination, $j = r$. Every entry in the array is in one of the three sets described by the invariant, and we have partitioned the values in the array into three sets: those less than or equal to x , those greater than x , and a singleton set containing x .

Quicksort: Performance (1)

- $$T(n) = T(q) + T(n - q + 1) + \Theta(n)$$
- Worst-case partitioning: $T(n) = T(n - 1) + \Theta(n)$:
$$T(n) = \Theta(n^2)$$
- Best-case partitioning: $T(n) = 2T(n/2) + \Theta(n)$:
$$T(n) = \Theta(n \log n)$$
- Average-case partitioning: Close to the best-case:
$$T(n) = \Theta(n \log n)$$

Quicksort: Performance (2)



A recursion tree for QUICKSORT in which PARTITION always produces a 9-to-1 split

Randomized QuickSort

RANDOMIZED-PARTITION(A, p, r)

```
1   $i = \text{RANDOM}(p, r)$   
2  exchange  $A[r]$  with  $A[i]$   
3  return PARTITION( $A, p, r$ )
```

RANDOMIZED-QUICKSORT(A, p, r)

```
1  if  $p < r$   
2       $q = \text{RANDOMIZED-PARTITION}(A, p, r)$   
3      RANDOMIZED-QUICKSORT( $A, p, q - 1$ )  
4      RANDOMIZED-QUICKSORT( $A, q + 1, r$ )
```

Analyzing Quicksort: Worst-case

- $T(n) = \max_{0 \leq q \leq n-1} (T(q) + T(n - q + 1)) + \Theta(n)$
- The guess is $T(n) = O(n^2)$; $T(n) \leq cn^2$
- $T(n) \leq \max_{0 \leq q \leq n-1} (cq^2 + c(n - q + 1)^2) + \Theta(n) : q = 0 \text{ or } q = n - 1$
- $T(n) \leq cn^2 - 2cn + c + \Theta(n) \leq cn^2$
- $T(n) = O(n^2)$

Analyzing Quicksort: Average-case (1)

- $T(n) = O(n + X)$; X is a total number of comparisons in line 4 of *Partition* over the entire execution of *QuickSort*.
- If A was sorted the elements will be $z_1, z_2, \dots, z_n$
 z_i is the i^{th} smallest element. $Z_{ij} = \{z_i, z_{i+1}, \dots, z_j\}$
- z_i and z_j might be compared at most 1 time when one of them is pivot.
- $X_{ij} = I\{z_i \text{ is compared to } z_j\};$

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}$$

Analyzing Quicksort: Average-case (2)

- $$E[X] = E \left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij} \right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$$
$$E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \Pr\{z_i \text{ is compared to } z_j\}$$
$$\Pr\{z_i \text{ is compared to } z_j\} = \frac{2}{j - i + 1}$$

Analyzing Quicksort: Average-case (3)

-

$$E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} = \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k+1}$$

$$E[X] < \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k} = \sum_{i=1}^{n-1} O(\log n) = O(n \log n)$$

$$T(n) = O(n + X) = O(n + cn \log n)$$

$$\mathbf{T(n) = O(n \log n)}$$

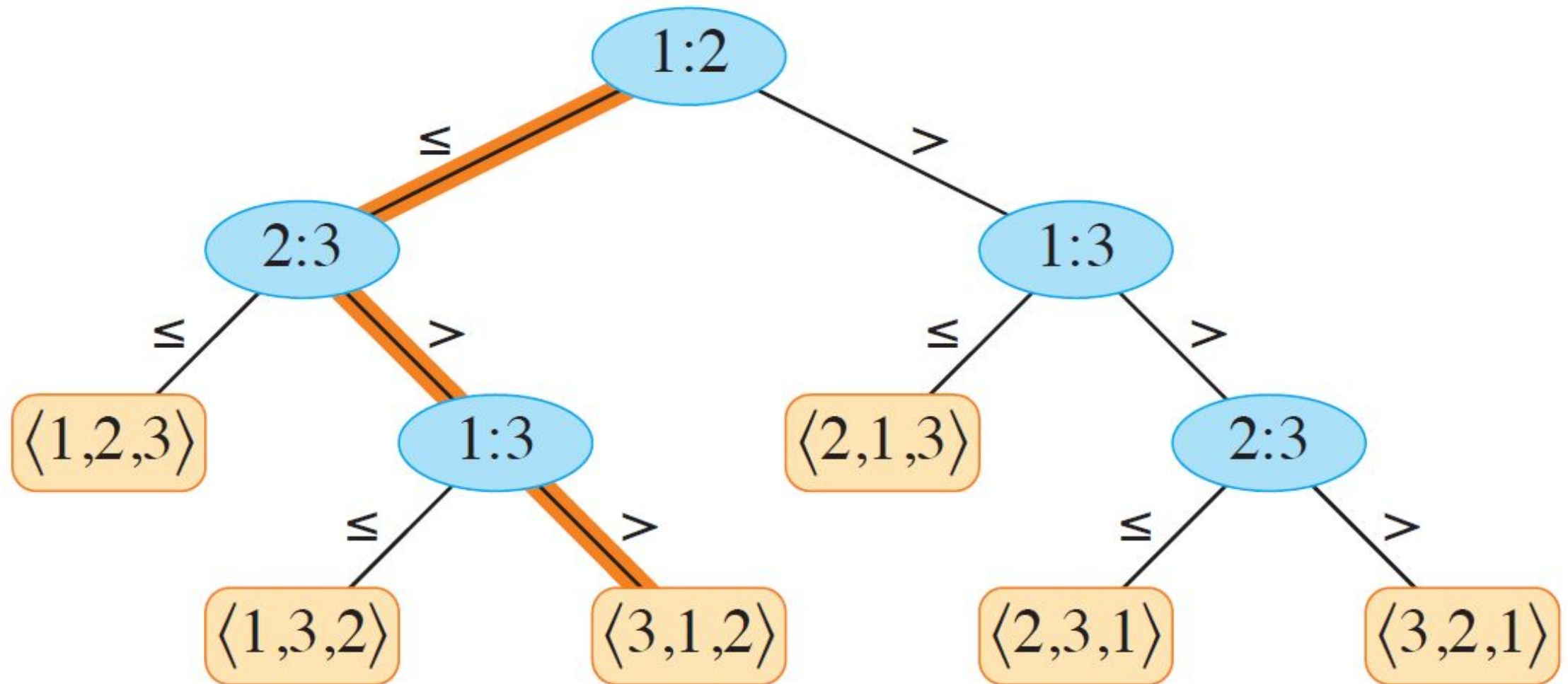
Comparison Sorts

-

Comparison sorts determine the sorted order based only comparisons between elements.

Any comparison sort must make $\Omega(n * \log n)$ comparisons in the worst case to sort n elements.

Lower Bounds for Sorting (1)



Lower Bounds for Sorting (2)

- A **decision tree** is a full binary tree that represents the comparisons between elements that are performed by a particular sorting algorithm operating on an input of a given size.
- The length of the longest simple path from the root of a decision tree to any of its reachable leaves represents the worst-case number of comparisons that the corresponding sorting algorithm performs.

Lower Bounds for Sorting (3)

- Any comparison sort algorithm requires $\Omega(n * \log n)$ comparisons in the worst case to sort n elements.
- Consider a decision tree of height h with l reachable leaves corresponding to a comparison sort on n elements.
 - $n! \leq l \leq 2^h$
 - $h \geq \log(n!)$
 - $h = \Omega(n * \log n)$

Counting Sort

-
- **Counting sort** assumes that each of the n input elements is an integer in the range 0 to k , for some integer k .
- When $k = O(n)$, the sort runs $\Theta(n)$

Counting Sort (1)

- Counting sort determines, for each input element x , the number of elements less than x . It uses this information to place element x directly into its position in the output array.
- Counting sort is a stable sort.

Counting Sort (1)

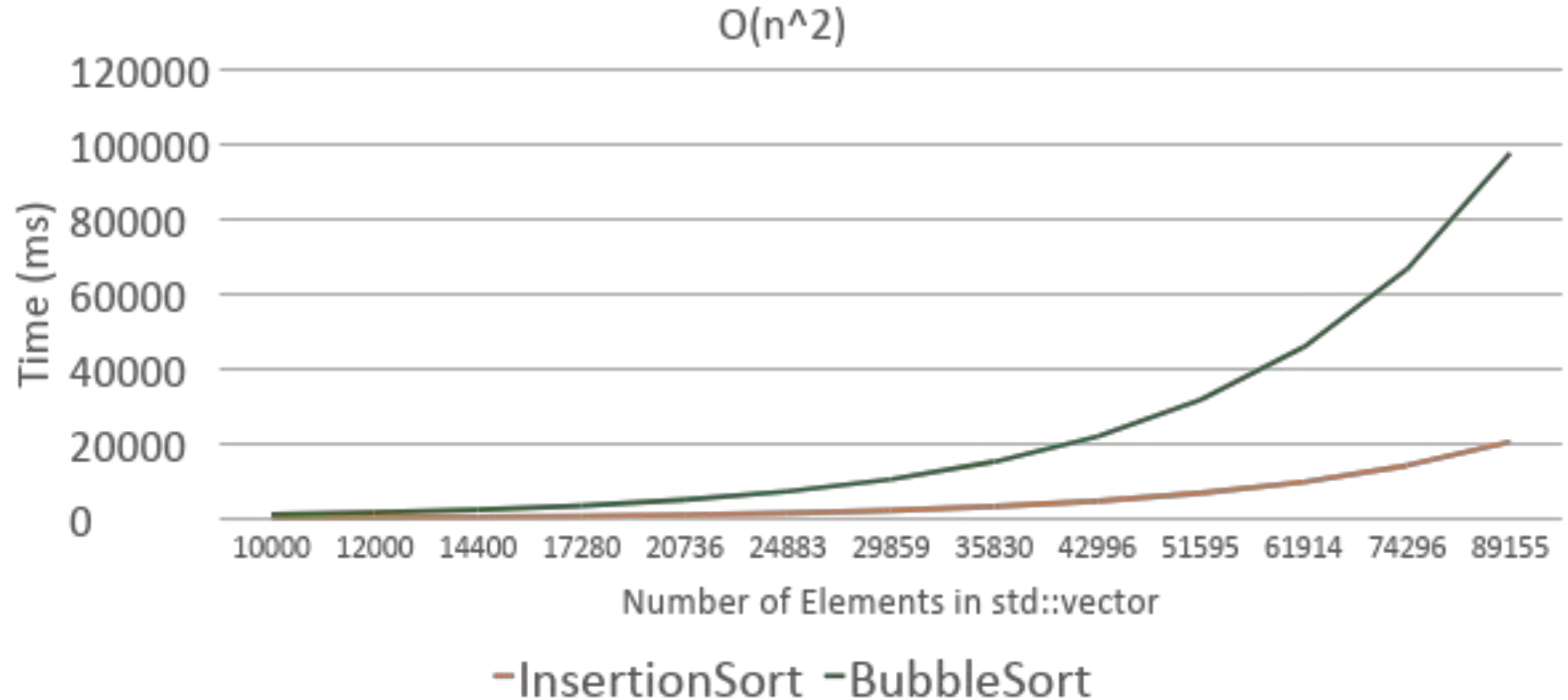
COUNTING-SORT(A, B, k)

```
1  let  $C[0..k]$  be a new array
2  for  $i = 0$  to  $k$ 
3       $C[i] = 0$ 
4  for  $j = 1$  to  $A.length$ 
5       $C[A[j]] = C[A[j]] + 1$ 
6  //  $C[i]$  now contains the number of elements equal to  $i$ .
7  for  $i = 1$  to  $k$ 
8       $C[i] = C[i] + C[i - 1]$ 
9  //  $C[i]$  now contains the number of elements less than or equal to  $i$ .
10 for  $j = A.length$  downto 1
11      $B[C[A[j]]] = A[j]$ 
12      $C[A[j]] = C[A[j]] - 1$ 
```

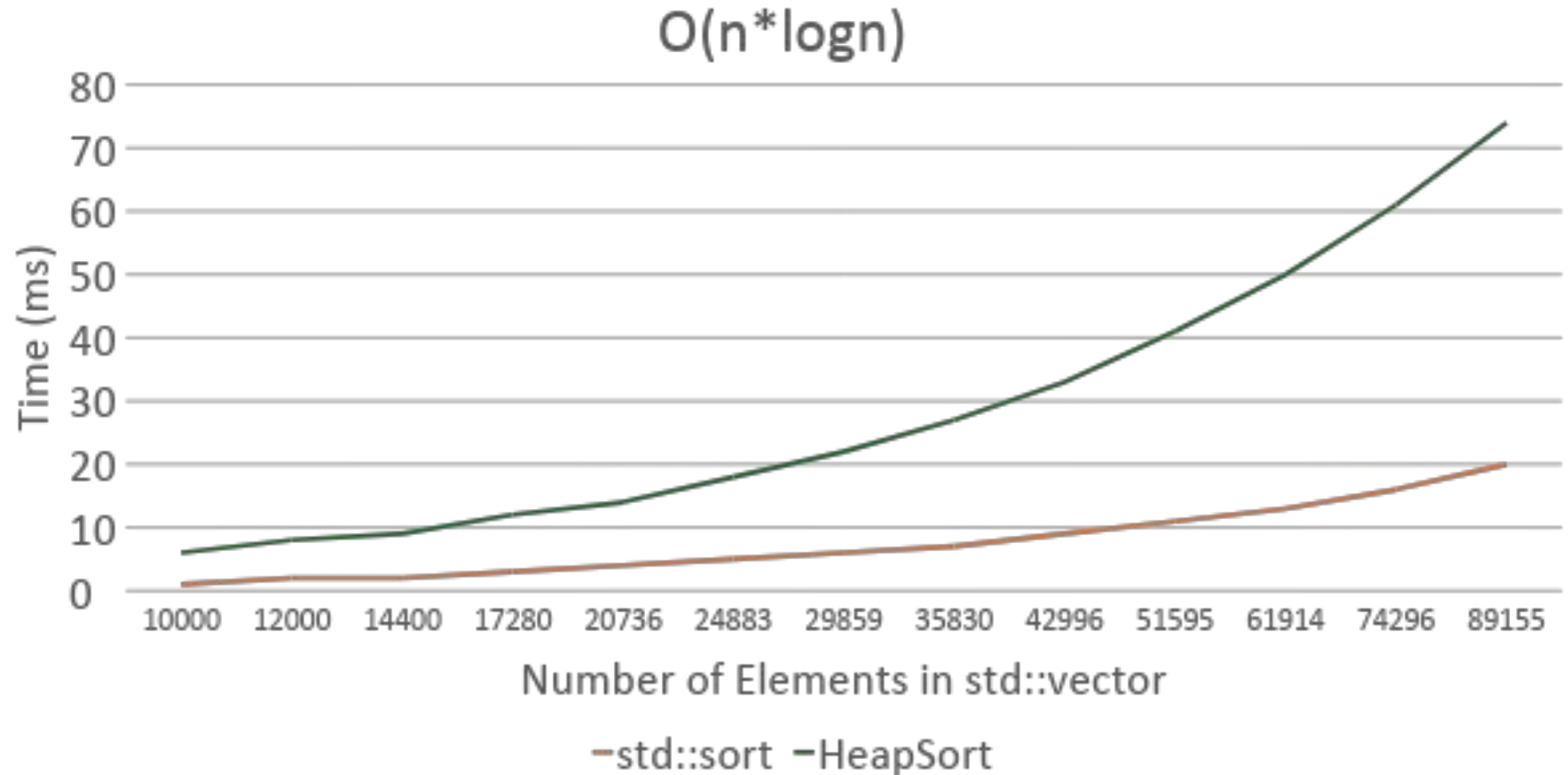
Analysing Sorting Algorithms

Insertion Sort, Bubble Sort, `std::sort`, Heap Sort

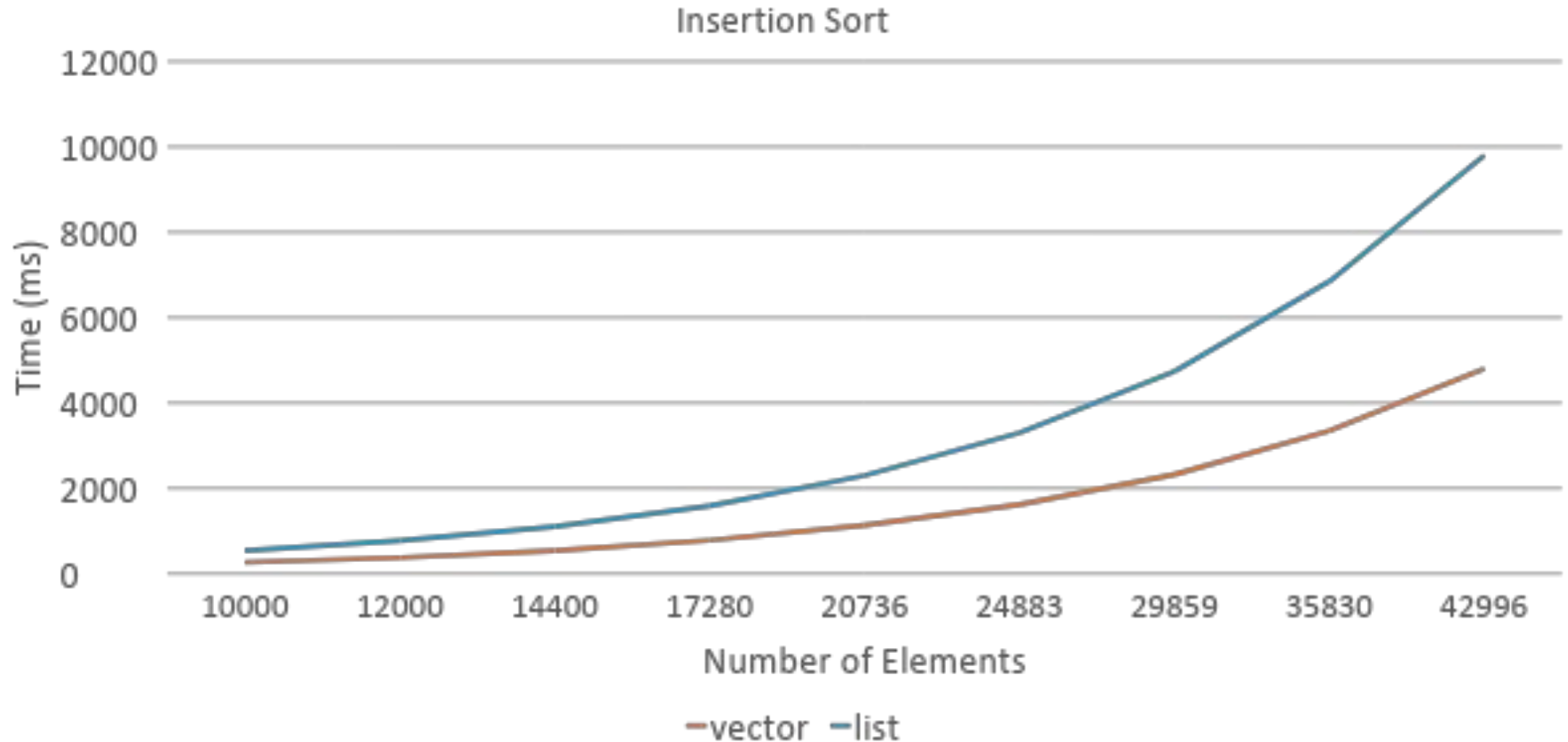
Runtime Comparison (1)



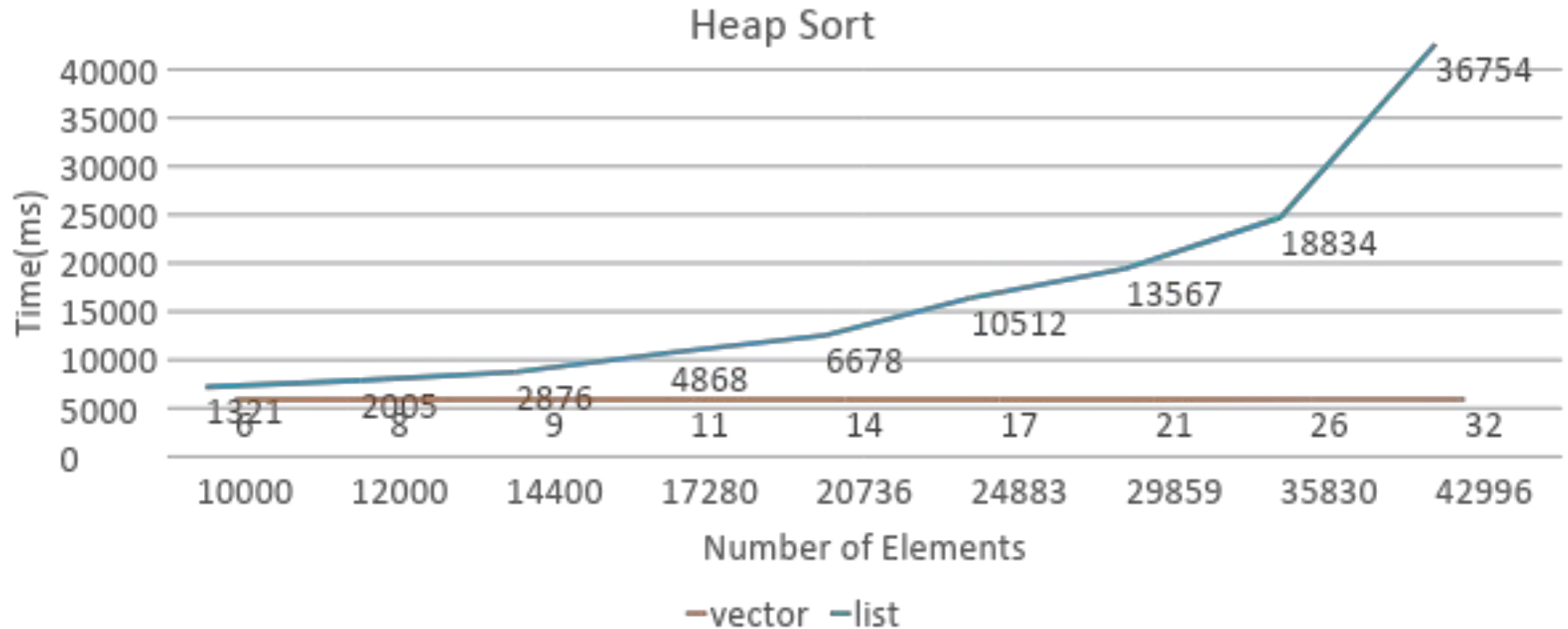
Runtime Comparison (2)



Insertion Sort Runtime Dependency on Data Structure



Heap Sort Runtime Dependency on Data Structure





Thank You